

Termin 1: Übungen

Bearbeiten Sie die Aufgaben im vorbereiteten Projekt und starten Sie es über ► **Run**. Verwenden Sie pro Aufgabe einen eigenen Programmlauf. Bei Eingaben setzen wir vorerst gültige Werte voraus.

Übungsblock 1: Setup und erste Änderungen

Aufgabe 1: Programm starten und verändern

Ziel: Das vorbereitete Projekt bearbeiten und ausführen.

Anforderungen:

1. Öffnen Sie die vorbereitete C#-Datei im Codespace und starten Sie sie über ► **Run**.
2. Ändern Sie einen ausgegebenen Text und ergänzen Sie eine weitere Textausgabe.
3. Speichern und starten Sie erneut. Prüfen Sie Inhalt und Reihenfolge der Ausgaben.

Tips: Eine Ausgabe lautet z. B. `Console.WriteLine("Hallo!");`. Achten Sie auf "...", (...) und ;.

Übungsblock 2: Datentypen und Variablen

Aufgabe 2: Passende Datentypen wählen

Ziel: Variablen mit passenden Typen anlegen, lesen und ändern.

Anforderungen:

1. Legen Sie Variablen an: Teilnehmerzahl `24`, Raum `"A204"`, Gruppenkennzeichen `'B'`, Temperatur `21.5` als `float`, Veranstaltung aktiv `true`.
2. Geben Sie jede Variable einzeln aus.
3. Weisen Sie der Teilnehmerzahl anschließend `25` zu und geben Sie sie erneut aus.

Tips: Nutzen Sie die Typentabelle. Für `float` ist das Suffix `F` erforderlich. Beim Ändern entfällt der Typ vor dem Namen.

Aufgabe 3: Gesamtpreis aktualisieren

Ziel: Mit Variablen rechnen und gespeicherte Werte ändern.

Anforderungen:

1. Legen Sie `anzahl` (`int`, Wert `3`) und `einzelpreis` (`double`, Wert `2.50`) an.
2. Speichern Sie `Anzahl × Einzelpreis` in `gesamtpreis` (`double`). Geben Sie ihn aus.
3. Erhöhen Sie `anzahl` um `2`.
4. Geben Sie den gespeicherten `gesamtpreis` erneut aus, ohne ihn neu zu berechnen.
5. Berechnen Sie `gesamtpreis` mit der neuen Anzahl neu und geben Sie ihn aus.

Tips: Alle Schritte in einem Programmlauf. Nutzen Sie `anzahl = anzahl + 2;`. Erwartete Ausgaben: `7.5`, `7.5`, `12.5` (ggf. mit Komma).

Aufgabe 4: Werte kopieren und ändern

Ziel: Werte zwischen Variablen übertragen und unabhängig ändern.

Anforderungen:

1. Legen Sie `punkte` als `int` mit dem Wert `12` an. Legen Sie `kopie` als `int` an und weisen Sie ihr den Wert von `punkte` zu.
2. Erhöhen Sie nur `punkte` um `5`. Geben Sie beide Variablen einzeln aus.
3. Weisen Sie `kopie` den aktuellen Wert von `punkte` zu. Geben Sie beide erneut aus.

Tips: Bei einer erneuten Zuweisung entfällt der Typ. Erwartete Ausgaben: `17, 12, 17, 17`.

Aufgabe 5: Artikel auf Kartons verteilen

Ziel: Ganzzahldivision und Restberechnung verwenden.

Anforderungen:

1. Legen Sie `artikel` mit dem Wert `23` und `proKarton` mit dem Wert `5` an, jeweils als `int`.
2. Berechnen Sie mit `/` die Anzahl vollständig gefüllter Kartons und mit `%` die übrigen Artikel. Speichern und geben Sie beide Ergebnisse aus.
3. Ändern Sie `artikel` auf `25`. Berechnen und geben Sie beide Ergebnisse erneut aus.

Tips: Zwei `int`-Werte ergeben bei `/` eine Ganzzahldivision. Erwartete Ausgaben: `4, 3, 5, 0`.

Übungsblock 3: Eingaben, Berechnungen und Entscheidungen

Aufgabe 6: Alter in Tagen berechnen

Ziel: Eine Eingabe umwandeln, rechnen und mit `if/else` entscheiden.

Anforderungen:

1. Lesen Sie das Alter in Jahren ein und wandeln Sie es mit `Convert.ToInt32(...)` in `int` um.
2. Berechnen Sie das ungefähre Alter in Tagen mit 365 Tagen pro Jahr und geben Sie es aus.
3. Geben Sie unter 18 Jahren `Minderjährig`, sonst `Volljährig` aus.

Tips: Testen Sie die Grenze: `17` → 6205 Tage / minderjährig; `18` → 6570 Tage / volljährig; `20` → 7300 Tage / volljährig.

Aufgabe 7: Gerade oder ungerade?

Ziel: Restberechnung und Vergleich als Bedingung verwenden.

Anforderungen:

1. Lesen Sie eine ganze Zahl ein.
2. Prüfen Sie mit `%` und `if/else`, ob die Zahl gerade oder ungerade ist.
3. Geben Sie das Ergebnis als Text aus.

Tips: `zahl % 2 == 0` prüft, ob kein Rest bleibt. Testfälle: `8` → gerade, `7` → ungerade, `0` → gerade.

Aufgabe 8: Zutritt prüfen

Ziel: Vergleiche und logische Operatoren in einer Entscheidung kombinieren.

Anforderungen:

1. Legen Sie `alter` als `int` sowie `hatTicket` und `stehtAufGaesteliste` als `bool` mit festen Startwerten an.
2. Erlauben Sie den Zutritt ab 18 Jahren, wenn ein Ticket vorliegt oder die Person auf der Gästeliste steht. Geben Sie andernfalls `Kein Zutritt` aus.
3. Testen Sie: 18 / true / false → erlaubt; 18 / false / true → erlaubt; 18 / false / false → abgelehnt; 17 / true / true → abgelehnt.

Tipps: Gruppieren Sie die Oder-Bedingung mit Klammern. Ergänzen Sie mit `!hatTicket` eine Ausgabe darüber, ob das Ticket fehlt.

Optionale Vertiefung

Aufgabe 9: Währungsrechner

Optionale Vertiefung

Ziel: Dezimaleingaben verarbeiten und Ergebnisse formatieren.

Anforderungen:

1. Legen Sie `double wechselkurs = 1.08;` als festen Übungskurs an.
2. Lesen Sie einen Euro-Betrag als `double` ein und berechnen Sie den Betrag in US-Dollar.
3. Geben Sie Eingabe und Ergebnis mit zwei Nachkommastellen aus.

Tipps: Verwenden Sie `Convert.ToDouble(...)` und z. B. `{dollar:0.00}` in einem interpolierten Text. 10 Euro ergeben 10.80 US-Dollar.

Aufgabe 10: Ticketpreis ermitteln

Optionale Vertiefung

Ziel: Mehrere Fälle mit `if-else if-else` unterscheiden.

Anforderungen:

1. Lesen Sie ein Alter ein.
2. Bestimmen Sie den Preis: unter 6 → frei; 6–17 → 5 Euro; 18–64 → 10 Euro; ab 65 → 7 Euro.
3. Geben Sie den passenden Preis aus.

Tipps: Prüfen Sie die Altersgrenzen in aufsteigender Reihenfolge. Testen Sie `5`, `6`, `17`, `18`, `64` und `65`.

Aufgabe 11: Schaltjahr-Rechner

Optionale Vertiefung

Ziel: Mehrere Teilbedingungen zu einer Entscheidung verbinden.

Anforderungen:

1. Lesen Sie eine positive Jahreszahl ein.
2. Ein Jahr ist ein Schaltjahr, wenn es durch 4, aber nicht durch 100 teilbar ist, oder wenn es durch 400 teilbar ist.
3. Geben Sie aus, ob das Jahr ein Schaltjahr ist.

Tipps: Verwenden Sie `%`, `==`, `!=`, `&&`, `||` und Klammern. Testfälle: 2000 und 2024 → ja; 1900 und 2023 → nein.

Aufgabe 12: Login-Simulation*Optionale Vertiefung*

Ziel: Textvergleiche und verschachtelte Entscheidungen anwenden.

Anforderungen:

1. Legen Sie `korrekterName = "admin"` und `korrektesPasswort = "uebung"` als `string`-Variablen an.
2. Lesen Sie einen Namen und ein Passwort ein.
3. Geben Sie aus: Name falsch → `Benutzername nicht gefunden.`; nur Passwort falsch → `Falsches Passwort.`; beide richtig → `Login erfolgreich!`.

Tipps: Vergleichen Sie mit `==`. Prüfen Sie das Passwort innerhalb des passenden Namensfalls mit einer weiteren `if/else`-Anweisung. Testen Sie alle drei Fälle.